



Scrum Development

¡Conviértete en un desarrollador profesional en Scrum!

Agile

Discover

Develop

Design

Test

0 Introducción

1 Conoce los fundamentos de Scrum

- 1.1 Equipo Scrum
- 1.2 Dueño del producto
- 1.3 Scrum Master
- 1.4 Pila de productos
- 1.5 Reunión de planificación de sprint
- 1.6 Scrum diario
- 1.7 Reunión de revisión de sprint
- 1.8 Sprint retrospectiva
- 1.9 Una introducción Scrum

2 Generación de equipos autogestionados

- 2.1 Tenga un buen objetivo
- 2.2 Proponga límites

3 Planificación del lanzamiento del producto

- 3.1 Planificación de lanzamientos y entrega predecible
- 3.2 ¿Por qué el software es tan impredecible?
- 3.3 Acepta la falta de previsibilidad
- 3.4 Centrarse en la calidad continua

4 Generación del valor del producto

- 4.1 El valor del producto
- 4.2 4 pasos para optimizar el valor del producto

5 Refinamiento de la cartera de productos

- 5.1 ¿Qué es el refinamiento de la cartera de productos?
- 5.2 ¿Por qué el refinamiento de varios equipos?
- 5.3 Formatos de refinamiento de varios equipos

6 Áreas de enfoque clave

- 6.1 Desarrollo de software emergente
- 6.2 Gestión de riesgos técnicos
- 6.3 Calidad continua
- 6.4 Integración continua (CI) / Entrega continua (CD)

7 Conclusiones

>>>INTRODUCCIÓN

En G-Talent estamos comprometidos con tu aprendizaje, es por eso que producimos diversos artículos, e-books, infografías, tutoriales, whitepapers y una infinidad de materiales que te permiten obtener conocimientos prácticos, sencillos y concisos de un tópico en particular.

Hoy te presentamos una guía con los aspectos más relevantes que debes conocer sobre Scrum Development.

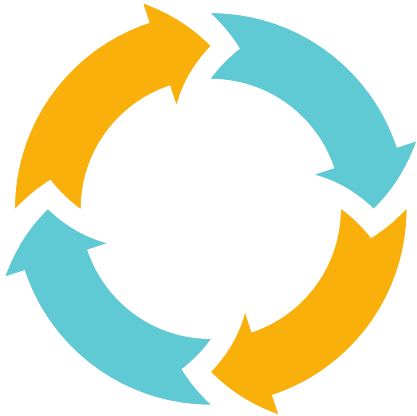
Como se describe en el marco de trabajo Scrum, los desarrolladores son las personas del equipo Scrum que están comprometidas a crear cualquier aspecto de un incremento utilizable en cada sprint.

La ruta de aprendizaje presente en esta guía se organiza por un conjunto de competencias profesionales de Scrum, Cada una de las cuales contiene una serie de áreas de enfoques que debes tener en cuenta para formar parte del equipo de desarrolladores en el marco de trabajo Scrum.

Esperamos sea de mucho provecho esta lectura.

G-Talent

>>>CONOCE LOS FUNDAMENTOS DE SCRUM



Scrum es un proceso ágil, comúnmente utilizado para el desarrollo de productos, especialmente el desarrollo de software. Scrum es un marco de gestión de proyectos que es aplicable a cualquier proyecto con fechas límites agresivas, requisitos complejos y un grado de singularidad. En Scrum, los proyectos avanzan a través de una serie de iteraciones llamadas sprints. Cada sprint suele durar de dos a cuatro semanas.

Una introducción a Scrum no estaría completa sin conocer los términos de Scrum que se utilizarán.

Equipo Scrum

Un equipo Scrum típico tiene entre cinco y nueve personas, pero los proyectos Scrum pueden escalar fácilmente a cientos. Sin embargo, Scrum puede ser utilizado fácilmente por equipos de una persona y, a menudo, lo es. Este equipo no incluye ninguno de los roles tradicionales de ingeniería de software, como programador, diseñador, probador o arquitecto. Todos en el proyecto trabajan juntos para completar el conjunto de trabajo que se han comprometido colectivamente a completar dentro de un sprint. Los equipos de Scrum desarrollan una forma profunda de camaradería y la sensación de que "estamos todos juntos en esto".

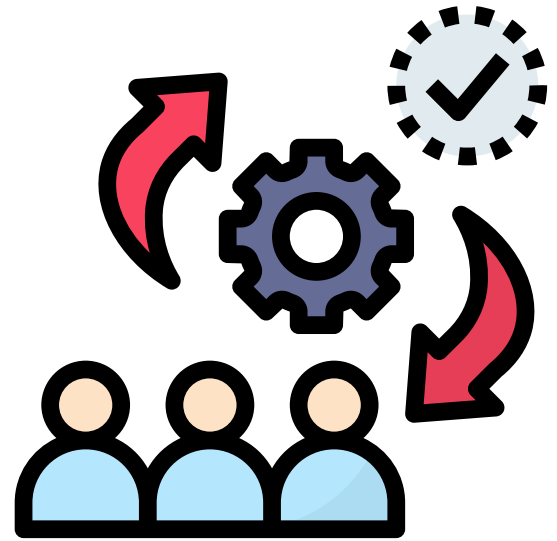


Dueño del producto

El dueño del producto es el actor clave del proyecto y representa a los usuarios, clientes y otros en el proceso. El propietario del producto suele ser alguien de la gestión de productos o del marketing, una parte interesada clave o un usuario clave.

Scrum Master

El Scrum Master es responsable de asegurarse de que el equipo sea lo más productivo posible. El Scrum Master hace esto ayudando al equipo a utilizar el proceso de Scrum, eliminando los impedimentos para progresar, protegiendo al equipo del exterior, etc.



Pila de productos

La pila de productos es una lista de características priorizadas que contiene todas las características o cambios deseados en el producto. Debes tener en cuenta que el término "acumulación" puede resultar confuso porque se usa para dos cosas diferentes:

- Para aclarar, la cartera de productos es una lista de características deseadas para el producto.
- Para el backlog del sprint, que es una lista de tareas que se deben completar en un sprint.



Reunión de planificación de sprint

Al comienzo de cada sprint, se lleva a cabo una reunión de planificación de sprint, durante la cual el propietario del producto presenta los elementos principales de la cartera de productos al equipo. El equipo de Scrum selecciona el trabajo que pueden completar durante el próximo sprint. Luego, ese trabajo se mueve de la lista de trabajos pendientes del producto a una lista de tareas pendientes, que es la lista de tareas necesarias para completar los elementos de la lista de productos pendientes que el equipo se ha comprometido a completar en el sprint.



Scrum diario

Cada día durante el sprint, se lleva a cabo una breve reunión llamada Scrum diario. Esta reunión ayuda a establecer el contexto para el trabajo de cada día y ayuda al equipo a mantenerse encaminado. Todos los miembros del equipo deben asistir al Scrum diario.

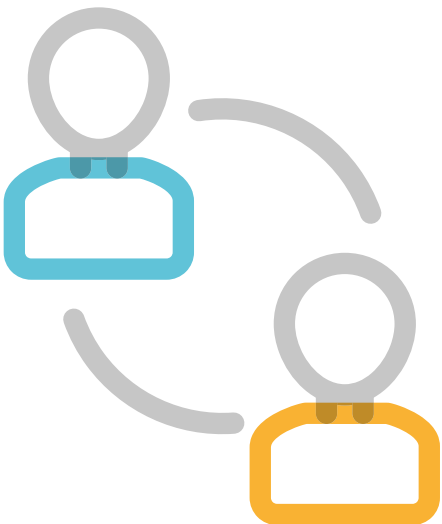


Reunión de revisión de sprint

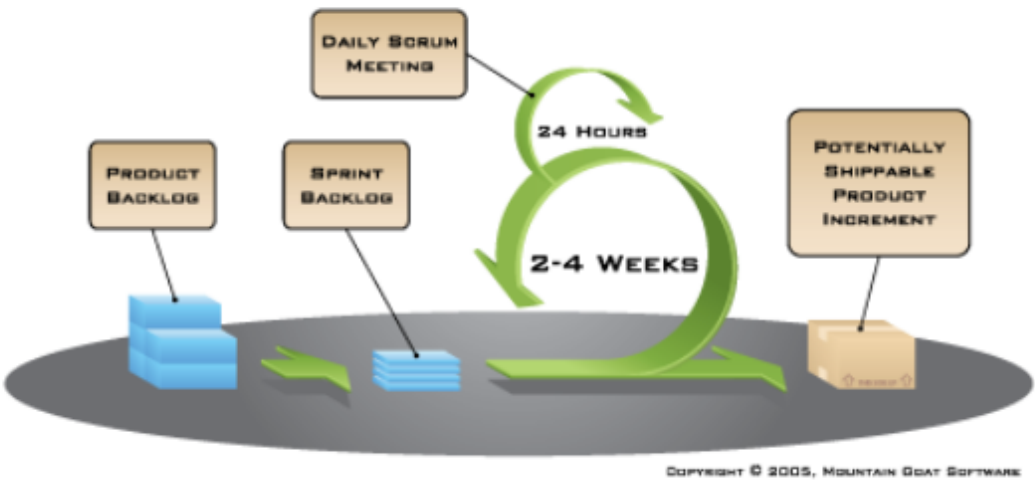
Al final de cada sprint, el equipo demuestra la funcionalidad completa en una reunión de revisión de sprint, durante la cual el equipo muestra lo que lograron durante el sprint. Normalmente, esto toma la forma de una demostración de las nuevas funciones, pero de manera informal; por ejemplo, no se permiten diapositivas de PowerPoint. La reunión no debe convertirse en una tarea en sí misma ni en una distracción del proceso.

Sprint retrospectiva

También al final de cada sprint, el equipo realiza una retrospectiva de sprint, que es una reunión durante la cual el equipo (incluido su Scrum Master y el propietario del producto) reflexiona sobre qué tan bien el Scrum está trabajando para ellos y qué cambios pueden desear.



Una introducción Scrum

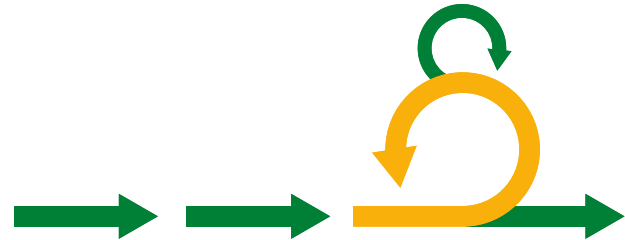


Este gráfico es una introducción a los elementos esenciales del uso de Scrum para el desarrollo ágil de software. A la izquierda, vemos la cartera de productos, que ha sido priorizada por el propietario del producto y contiene todo lo deseado en el producto que se conoce en ese momento. Los sprints de dos a cuatro semanas se muestran con un círculo verde más grande.

Al comienzo de cada sprint, el equipo selecciona una cierta cantidad de trabajo de la cartera de productos y se compromete a completar ese trabajo durante el sprint. Parte de averiguar cuánto pueden comprometerse es crear el backlog del sprint, que es la lista de tareas (y una estimación de cuánto tiempo tomará cada una) necesarias para entregar el conjunto seleccionado de elementos del backlog del producto que se completará en el sprint.



Al final de cada sprint, el equipo produce un incremento de producto potencialmente entregable, es decir, software funcional de alta calidad. Cada día, durante el sprint, los miembros del equipo se reúnen para discutir su progreso y cualquier impedimento para completar el trabajo para ese sprint. Esto se conoce como Scrum diario y se muestra como el círculo verde más pequeño arriba.



>>>GENERACIÓN DE EQUIPOS AUTOGESTIONADOS



Los equipos dirigidos por gerentes dejan a los miembros del equipo solo la autoridad para la ejecución de tareas, mientras que los gerentes monitorean y administran su trabajo; los equipos autogestionados tienen la autoridad para decidir de manera autónoma quién hace qué, cuándo y cómo.



La autogestión no se puede imponer a los equipos. Nadie puede establecer la autogestión ni ordenar que los equipos se autogestionen. La autogestión debe surgir proporcionando buenos objetivos atractivos y límites de apoyo.

El grado, la dirección y la intensidad de la autogestión se pueden influir y fomentar manipulando esos dos ingredientes cruciales.

Ambos ingredientes son necesarios para darle al equipo el marco y la dirección en las que operarán los colaboradores. Así ambos ingredientes dan orientación sobre qué lograr y aclaran el espacio de la solución.

Tenga un buen objetivo

Una buena meta motiva y da contexto sobre el propósito y las razones por las que el equipo está tratando de lograrla. Alinea al equipo, ayuda con la planificación y deja espacio para que el equipo decida cómo lograr el objetivo. Un buen objetivo también ayuda a mantener el rumbo en caso de que ocurran distracciones.

Si no se fija ningún objetivo, los equipos no saben la dirección en la que deben partir y se pierden. Se sumergen de cabeza en algo, no progresan mucho mientras luchan por medir y, a menudo, fallan porque no saben cuándo han terminado.

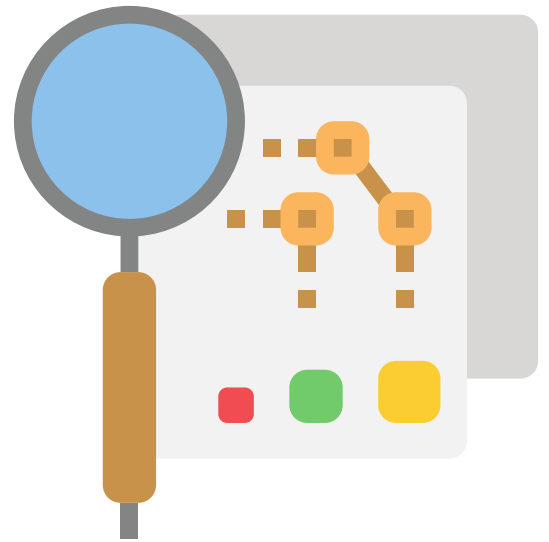
Un objetivo también ayuda a comunicarse y darse cuenta de que el equipo ha terminado.



En Scrum, esta es la razón por la que el equipo Scrum trabaja hacia un Objetivo de Producto y acuerda un Objetivo de Sprint en cada Sprint, durante la Planificación del Sprint.

El objetivo del producto proporciona el contexto, el porqué de la iniciativa de desarrollo y proporciona un enfoque para el equipo Scrum hacia un objetivo valioso más amplio. Cada sprint debería acercar el producto al objetivo general del producto.

El Sprint Goal proporciona el objetivo a corto plazo de los elementos del Product Backlog entregados en el sprint. Da la razón por la que el equipo Scrum está construyendo los elementos de la Pila de Producto seleccionados en el Incremento del sprint. El Sprint Goal ayuda al equipo a decidir qué hacer cuando surge un trabajo no descubierto durante el sprint.



Un buen Sprint Goal es entendible, concreto y medible. La lista de verificación de Jurgen Appelo para metas ágiles y la plantilla Sprint Goal de Roman Pichler pueden ayudar a los equipos Scrum a crear una meta de sprint concreta y medible.

Una verificación de confianza, formulada como una pregunta de escala de cinco en cinco durante la planificación del sprint, puede ayudar a mejorar aún más la confianza y la comprensión del equipo.



Un buen Sprint Goal es visible para el Scrum Team, a menudo colocado en el tablero Scrum en el espacio del equipo. Al referirse a un Sprint Goal visible, el Scrum Team puede evaluar su progreso y decidir las próximas acciones para alcanzarlo en el Daily Scrum.

Lo mismo es válido para el objetivo del producto. Como el objetivo del producto es parte de la cartera de productos, es muy visible para el equipo Scrum. Esto ayuda al equipo a mantenerse claro y concentrado durante el refinamiento, cuando dividen el objetivo a largo plazo en partes más pequeñas de trabajo valioso para los próximos sprints.

Proponga Límites

Además de las metas, los equipos necesitan límites para autogestionarse. Los equipos sin límites (o demasiado vagos) carecen de alineación y tienen demasiadas posibilidades.

Los límites demasiado restrictivos reducen la autogestión, ya que restringen la creatividad y la autonomía; de ahí que a menudo conduzcan a una falta de motivación y compromiso en el equipo.

Los límites que fomentan la autogestión son de apoyo. Son sencillos. Todos en el equipo los conocen. Se aceptan porque todo el mundo ve el valor y el propósito subyacentes. Esos límites están bien equilibrados, brindan orientación y dejan autonomía al equipo al mismo tiempo. Esos límites construyen y respaldan un marco confiable y seguro en el que la información, la opinión y el conocimiento se comparten abiertamente entre los miembros del equipo, para que la colaboración creativa sea posible.



Con su conjunto minimalista de reglas, Scrum Framework en sí mismo es un conjunto de límites bien equilibrado. Sus reglas están dirigidas a permitir el empirismo, la creatividad y la autonomía, promoviendo así la autogestión dentro del equipo Scrum.

Los Scrum Events y Scrum Artifacts brindan transparencia y retroalimentación rápida para que el equipo Scrum pueda inspeccionar los resultados con frecuencia y adaptarse si los resultados están fuera de la tolerancia aceptable.

Los roles de Scrum separan las responsabilidades en qué hacer (Product Owner), cómo hacer el trabajo (desarrolladores) y el marco (Scrum Master).



Timeboxing asegura que el Scrum Team se mantenga enfocado, sincronizado dentro de los miembros de su equipo y en línea con sus partes interesadas.

La Definición de Terminado crea transparencia y confianza sobre lo que se debe hacer para que el Incremento sea entregado al cliente. El enfoque de los valores de Scrum, el respeto, la apertura, el compromiso y el coraje sirven para generar confianza como base clave para la colaboración. Scrum Framework define estos límites y al mismo tiempo deja mucho espacio para que el equipo Scrum elija cómo aplicar exactamente las reglas. A menudo, el equipo Scrum elige implementar estrategias y tácticas adicionales para optimizar su modelo de trabajo.

Hay más límites para los equipos: presupuesto, organización, administración, límites del mercado y otros.



A menudo se ven equipos que trabajan en entornos con límites muy restrictivos que obstaculizan al equipo en su autonomía y flexibilidad y, por lo tanto, en su capacidad de autogestión.

Como Scrum Master no podemos imponer la autogestión a los equipos, pero podemos trabajar en los límites que restringen la capacidad de los equipos para autogestionarse.



>>>PLANIFICACIÓN DEL LANZAMIENTO DEL PRODUCTO

Planificación de lanzamientos y entrega predecible



La incompatibilidad entre la entrega predecible y la agilidad es ficticia, y aunque generalmente es creada por una organización y estructura que no está dispuesta a dejar de lado las viejas formas y abrazar a los inquilinos de ágil, también puede ser el resultado de un Scrum Team. Existe una falta de comprensión de que la agilidad y el camino hacia la agilidad es mucho más que un simple cambio en la forma en que construye software, es un cambio fundamental en la forma en que maneja su negocio. Al igual que el movimiento esbelto en la fabricación, las empresas que lo adoptaron de todo corazón fueron las que finalmente vieron la ventaja competitiva que proporciona. Si uno no está dispuesto a abandonar las viejas costumbres, no podrá alcanzar el valor de las nuevas.

Este cambio requerirá trabajo duro y coraje, ya que la transparencia fundamental requerida para inspeccionar y adaptarse de manera efectiva está en desacuerdo con las medidas del pasado. La falta de previsibilidad del desarrollo de software es la clave para comprender el nuevo modelo.



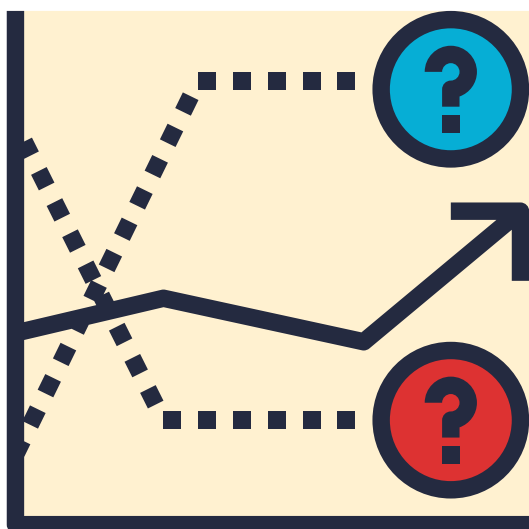
¿Por qué el software es tan impredecible?

Todo el desarrollo de software es desarrollo de productos. En la manufactura esbelta, podemos optimizar la producción de productos predesarrollados a través de la naturaleza de su producción predecible. Cada unidad de trabajo requiere la misma cantidad de materiales y tiempo para producir, por lo que cualquier cambio que hagamos en el proceso, el tiempo o los materiales se puede calificar fácilmente y demostrar el beneficio. La fabricación vive en el mundo predictivo.



Con el software, todo lo que creamos toma su propia cantidad de tiempo: realmente solo puede saber cuánto tiempo tomó algo después de que se completó. Incluso en la industria manufacturera, si le preguntara a un ingeniero cuánto tiempo tomaría desarrollar un nuevo tipo de unidad de trabajo, no podría decirle con certeza. Sin embargo, una vez que lo hayan desarrollado, pueden decirle exactamente cuánto tiempo llevará hacer cada uno y luego optimizar sistemáticamente el proceso que utiliza para hacerlo. En el desarrollo de software siempre estamos diseñando nuevos productos, por lo tanto, no tenemos certeza... y esto a menudo resulta en caos. El software vive en el mundo empírico.

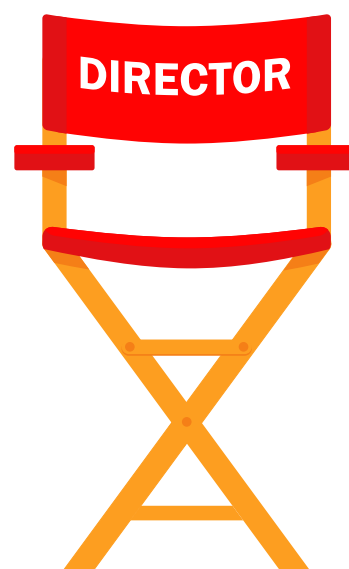




Sin embargo, no todo está perdido, ya que podemos, al mirar nuestro historial de entregas para cosas similares, hacer un pronóstico bastante bueno.

Entonces, lo mejor que podemos hacer es esforzarnos para que ese pronóstico sea lo más preciso posible, al tiempo que aceptamos que más tiempo dedicado a la planificación no necesariamente afecta la precisión de ese pronóstico.

En última instancia, el desarrollo de software es un esfuerzo creativo y tiene la misma falta de previsibilidad que tiene pintar una imagen, escribir un libro o hacer una película. Sin embargo, las películas se hacen todo el tiempo. ¿Cómo puede ser eso posible? Bueno, tienen un director (Product Owner) que tiene un montón de dinero y un plan de entrega, un productor (Scrum Master) para asegurarse de que todos tengan las habilidades, el conocimiento y los activos disponibles en el momento y lugar correctos y una o más unidades (Scrum Teams) que tengan todas las habilidades necesarias para convertir las ideas de los directores en una película funcional. Crean guiones gráficos de lo que esperan crear para poder ejecutarlo entre las partes interesadas y obtener comentarios. Llevan esos guiones gráficos a las unidades que trabajan en colaboración con los equipos de acrobacias, utilería, iluminación, cámara, sonido y vestuario para obtener estimaciones y costos y, en última instancia, coordinar para crear la película. A veces no saben cómo hacer las cosas y tienen que intentarlo y ver qué obtienen.

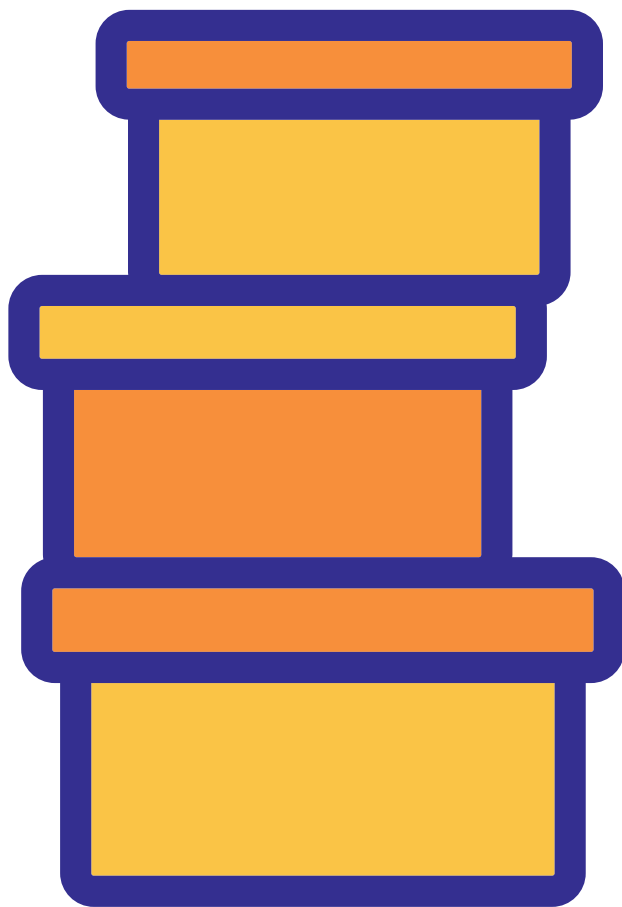


Hacer una película es como crear software, necesitas un presupuesto, necesitas un plan y estás tratando de llegar a una fecha de envío. Al igual que hacer una película, tienes que ganar dinero al final del día para poder hacerlo todo de nuevo.

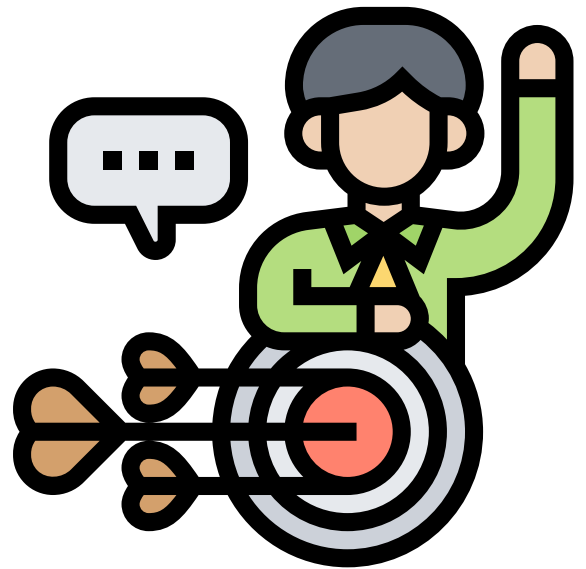


Acepta la falta de previsibilidad

Si bien la falta de previsibilidad es parte de la naturaleza de la creación de software, hay muchas cosas que podemos hacer para disminuir el impacto de ese caos. De hecho, si tuviera que estimar todas las cosas discretas que necesita hacer para lograr un objetivo (llamémoslas elementos de la cartera de pedidos) pequeño, mediano y grande, ¿cuál sería su desviación estándar de las horas reales? Apostaría a que es bastante grande. Tan grande, de hecho, que al menos la mitad de todos los medios se clasificarían con mayor precisión como grandes. Pero esa reclasificación solo se puede hacer en retrospectiva. Este es de hecho uno de los inquilinos del movimiento No-Estimates, ya que en realidad solo hay tres clasificaciones de tamaño: pequeño y cabe en un sprint, o demasiado grande para caber en un sprint.



Esta dificultad de estimación es normal para las organizaciones que avanzan hacia la agilidad, ya que la transparencia que aporta descubre este tipo de problemas. Para aumentar la precisión de nuestros pronósticos, hay una serie de actividades simples que podemos realizar. Estas actividades, aunque fáciles de entender, son muy difíciles de realizar, ya que requieren un cambio de cultura dentro de su organización, así como el coraje de los participantes para hacerlas funcionar.



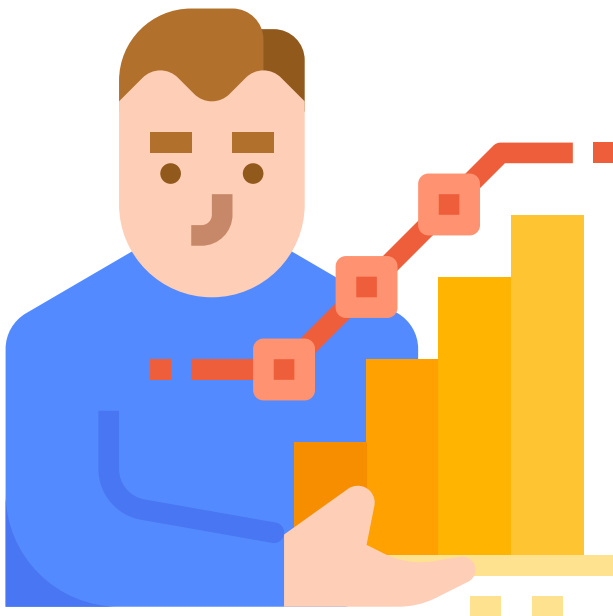
Centrarse en la calidad continua



La mayor parte del software carece de calidad por la sencilla razón de que no se puede ver fácilmente la calidad en el software como lo haría con una mesa o una pintura. No me refiero a la calidad de la interfaz de usuario, sino a la calidad oculta; la calidad del código.

Si presiona a los desarrolladores para que entreguen, reducirán continuamente y cada vez más la calidad para cumplir con los plazos.

La falta de calidad del código da como resultado un aumento de la Deuda Técnica (o más exactamente un fondo sin cobertura) que a su vez da como resultado dos desagradables. La primera es que los equipos tienen que pasar cada vez más tiempo luchando con la complejidad de su software en lugar de con las nuevas funciones.



La única forma de manejar la deuda técnica es dejar de crearla y luego pagar un poco cada iteración. Sin embargo, si está tan ahogado en una deuda técnica que no puede crear un software que funcione al final de la iteración, entonces:

- Cree una breve lista de verificación medible que refleje el producto mínimo liberable (Definición de Listo)
- Deje de agregar nuevas funciones y haga que su producto cumpla con esa lista de verificación y lance su producto.
- Mientras tenga un incremento de software en funcionamiento (Sprint)
- Trabajar para crear algo de valor (Incremento)
- Trabajar hacia una nueva meta mientras se cumple con el DOD (Sprint Goal)



- Deje las cosas mejor de lo que las encontró (Excelencia en ingeniería)
- Revise esa cosa de valor con sus partes interesadas (Revisión de Sprint , Adaptación del Backlog)
- Obtenga comentarios sobre al menos una cosa nueva para las partes interesadas.
- Actualice el Backlog para reflejar esta nueva información.
- Reflexiona sobre cómo trabajaste con todo tu equipo (Sprint Retrospective)
 - ¿Está aumentando la calidad?
 - ¿Está aumentando el DOD?
 - ¿Qué podemos cambiar para mejorar las cosas?

Puede llamar a la actividad que resulta de abandonar el ciclo while del software en funcionamiento como Scrumble; Debe dejar de acumular más funciones sobre las que no funcionan y arreglar las cosas para que pueda crear cosas nuevas. En última instancia, los equipos profesionales crean software que funciona.

Hay una serie de estrategias que pueden ayudarlo a dejar de crear y comenzar a pagar la deuda técnica:



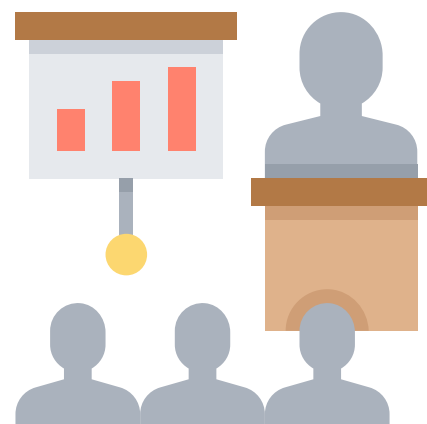
Requisitos suficientes:

I
N
V
E
S
T

Si su cartera de pedidos tiene cosas que son demasiado grandes o demasiado vagas, su equipo no podrá realmente entenderlas y esto, a su vez, crea un multiplicador de incertidumbre. Siga el modelo INVEST (independiente, negociable, valorado, estimado, pequeño, comprobable) para cada cosa que le pida al equipo que entregue. Si invierte en su cartera de pedidos de esta manera, le resultará mucho más fácil entregar los contenidos y, por lo tanto, predecir esa entrega. Esto requerirá que dediques una cantidad significativa de tiempo a perfeccionarlo. El refinamiento del Backlog es clave para facilitar un flujo de Elementos del Backlog procesables para su equipo.

Los desarrolladores eligen lo que pueden entregar: esto implica que los desarrolladores pueden rechazar cualquier elemento del trabajo pendiente que no comprendan.

Si aceptamos que cada Desarrollador está tratando de hacer todo lo posible para cumplir con su Product Owner, entonces la única razón para rechazar algo sería si un artículo es demasiado grande o no tiene suficientes detalles para comprender. Estos elementos de la lista de trabajos pendientes se pueden poner en la cola para mejorarlos y perfeccionarlos durante el siguiente Sprint. Recuerde que no existe un elemento de la cartera de pedidos rechazado, solo comentarios procesables y mejora continua.



Definición de Terminado (DoD):

Además de tener requisitos suficientes, el mayor obstáculo para la previsibilidad es la falta de comprensión común de HECHO. Hecho para desarrolladores debe igualar lo que significa completar un artículo sin que se requiera más trabajo para enviarlo. Si no puede enviar un software que funcione, debe dejar de correr, hacer Scrumble y concentrarse en poner su software en una forma que pueda entregarse en un Sprint.



Prueba primero: Céntrate en las prácticas de prueba primero, como TDD o ATDD, para asegurarte de que tus ingenieros no solo hayan creado lo que esperan, sino que, en última instancia, tú hayas creado lo que el cliente espera.

Iteraciones de longitud fija:

Si tiene iteraciones de longitud variable, no puede estar seguro de lo que puede hacer en un período de tiempo en particular. ¿Cuánta descomposición necesita hacer con el trabajo pendiente? ¿Cuánto puede entregar el equipo en una sola iteración? No puede estar seguro a menos que tenga una iteración de longitud fija y rechace la idea de iteraciones escalonadas.



Sin equipos separados:



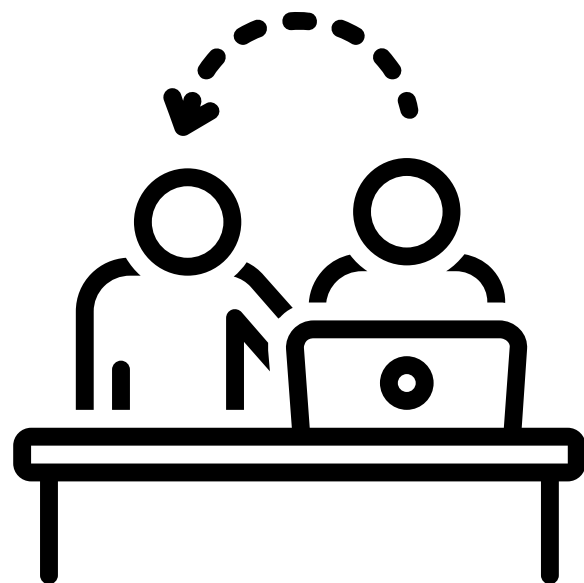
Esto significa que no hay equipos de prueba separados, equipos de administración de configuración y definitivamente no hay equipos de mantenimiento separados. Es difícil de entender para la gente, especialmente con el enfoque reciente en DevOps, pero si tiene equipos separados, ¿Por qué sus Desarrolladores, los que están en mejor posición para solucionar cualquier problema, se preocupan por los problemas de otros equipos? Las organizaciones más exitosas en la creación de software tienen equipos de desarrollo que poseen todo el ciclo de vida de la aplicación.

Administrar dependencias:

Administrar las dependencias es una tarea difícil, lo ideal sería siempre minimizar la cantidad de dependencias que tiene. Los desarrolladores deben tener todas las habilidades necesarias para ofrecer lo que desea con el nivel de calidad que desea. Entonces, si necesita tener bases de datos en producción o secuencias de comandos para la entrega de producción, es posible que necesite un administrador de bases de datos o un administrador de operaciones o dos.



Esto puede ser difícil para muchos equipos u organizaciones, pero tendrá mucho menos éxito creando silos como Configuration Management. Más bien, agregue a las personas que necesita al equipo. Sin embargo, si depende de un equipo separado, tal vez tenga una aplicación de la que dependen todas las demás aplicaciones, es posible que necesite otra forma. Esto no es un silo de tipos de habilidades individuales, sino de un dominio y ese equipo simplemente tiene algo en su acumulación de lo que usted depende. Depende de los respectivos propietarios de productos del equipo luchar para negociar cuándo se harán estas cosas.



>>>GENERACIÓN DEL VALOR DEL PRODUCTO

El valor del producto



En todo lo que hacemos debe haber algún valor, cierta importancia en lo que hacemos. Si no es así, ¿por qué hacerlo?

Sin ningún valor, lo que estás haciendo se convierte en una tarea o puedes pasar fácilmente a otra cosa. Por eso, necesitamos ver el valor de lo que estamos haciendo.

¿Por qué?

Sin embargo, como equipo, no siempre vemos el valor al mismo tiempo; dependiendo de su punto de vista verá el valor en diferentes etapas.

En un proyecto hay mucha gente involucrada y todos están buscando el "Valor", es decir, ¿Qué hay para ellos?

Un desarrollador verá el valor cuando cree los cambios en los que se le pidió que trabajara. Sin embargo, ¿siempre hay valor en el trabajo que han realizado? - No si tiene buggy y nadie lo usa.

Un Scrum Master verá el valor cuando vean al equipo trabajando como un equipo autoorganizado, con las historias moviéndose a "Listo".

Un propietario de producto verá el valor cuando se realice el lanzamiento y podrá explicar a las partes interesadas qué mejoras se han realizado.

Para las partes interesadas, ven el valor cuando más clientes utilizan el producto, ya que es el líder del mercado.



Como puede ver, para cada una de las personas involucradas en el proceso el valor llega en diferentes momentos. Sin embargo, es solo cuando el producto llega al cliente que se alcanza el valor real - hasta ese momento estamos cargando "Valor Deuda".

¿Por qué consideramos la importancia del calendario y el presupuesto al evaluar el éxito de un proyecto? ¿Por qué ponemos tanto énfasis en los plazos del proyecto y en cumplir con las conjeturas hechas al inicio del proyecto? Ahí no es donde está el valor como hemos visto en el ejemplo anterior.

Escalas de tiempo son importantes, pero no son la cosa más importante. Son un indicador, un atributo del proyecto, pero una vez que se han logrado, no aportan más valor .

A menos que exista una razón específica en el momento, ningún Cliente comprará algo debido a que el producto está disponible a tiempo si la funcionalidad/el valor no está allí.



4 pasos para optimizar el valor del producto

Paso 1:

Fomentar una mentalidad de producto (sobre una mentalidad de proyecto)

Una mentalidad de producto consiste en centrarse en la creación de resultados valiosos.

La salida no importa si está construyendo cosas que la gente no quiere o no usará.

La medida del éxito generalmente se basa en el triángulo de hierro: entregar todo el alcance definido a tiempo y dentro del presupuesto.

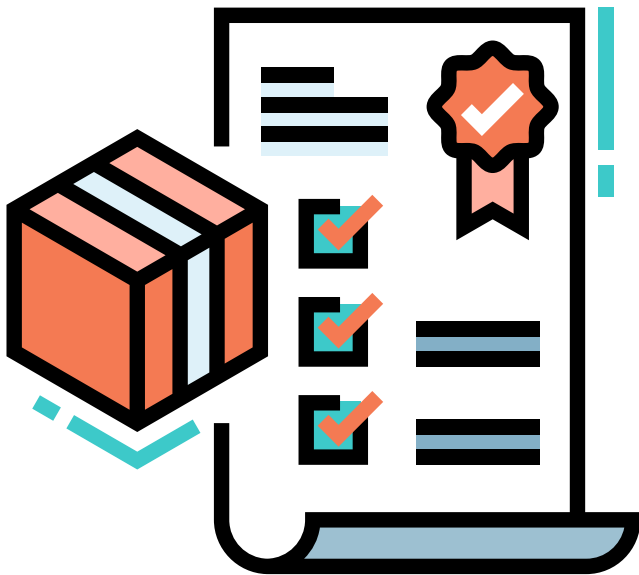


Scrum no se trata de entregar más cosas más rápido y más barato.

Scrum se trata de ofrecer un mayor valor con más frecuencia.

Y al hacer esto, reduce el riesgo para la organización y aprovecha el arte de lo posible.

Si bien los presupuestos y los horarios aún tienen un propósito, debe poner más énfasis en asegurarse de que está construyendo las cosas correctas. Los siguientes pasos proporcionarán detalles sobre cómo hacer esto. Sin embargo, siempre debe hacer el trabajo de fomentar una mentalidad de producto.



Paso 2: pinta la imagen más amplia

Dado que está construyendo de manera iterativa e incremental, es esencial tener una idea clara sobre el panorama general de lo que está trabajando y por qué. Esto ayuda a determinar si todavía está alineado e informa las adaptaciones necesarias.

Existen muchas técnicas para ayudar a aclarar lo que el producto está tratando de lograr (visión del producto) y el modelo de negocio detrás de él. Una visión de producto comunica qué desea que sea este producto y para quién. Habla con el cliente objetivo y la propuesta de valor principal proporcionada a ese cliente.

El panorama más amplio también incluye la definición de valor. Habrá muchas características y funciones deseadas. Para tomar decisiones sobre qué es más valioso o menos valioso, debe definir las facetas de valor más importantes para el producto y cómo sabrá que logró el valor deseado.



Paso 3: Habilitar la aparición de valor

Las soluciones a problemas complejos surgirán a medida que realiza el trabajo (no solo al analizar y hablar sobre el trabajo). Podrías aprender que las suposiciones estaban equivocadas. Podía ver que el mercado o incluso el modelo de negocio ha cambiado.

Por lo tanto, debe permitir que surja valor a medida que construye el producto. El Product Backlog representa lo que planea construir y el orden en el que planea hacerlo.

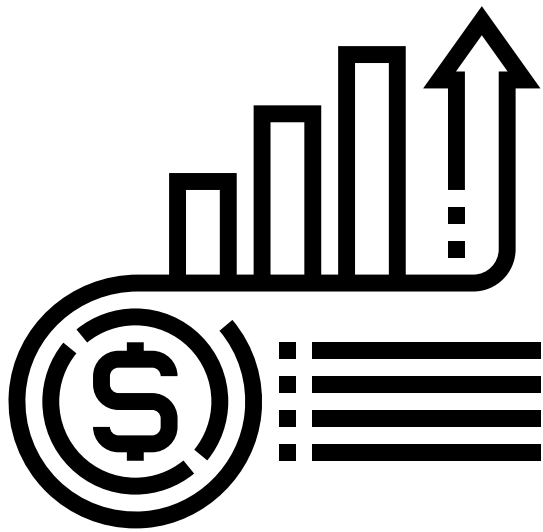
Paso 4: validar el valor real

Ahora tiene los componentes básicos en su lugar, por lo que puede medir el valor real.

El valor es solo una suposición hasta que el mercado lo valide.

Debe liberar para obtener la imagen más precisa del valor real entregado. Busque datos empíricos para crear la transparencia necesaria para una toma de decisiones informada.

¿Cuál es el valor real obtenido frente al valor deseado? ¿Cómo recopilará estos datos empíricos de manera eficaz?



Los equipos de desarrollo a menudo pueden ayudar a encontrar formas de incorporar capacidades de recopilación de datos en el producto. A medida que un producto crece en tamaño y complejidad, necesitará hacer crecer sus procesos y herramientas para recopilar estos datos empíricos.

Una vez que tenga los datos, puede analizar las tendencias. Recuerde que un punto de datos en el tiempo no le dice mucho. Las tendencias son lo que importa. Y necesita varios tipos de datos porque un tipo de datos no le dice la historia completa, ya que a menudo hay muchos factores (algunos fuera de su control) que pueden afectar el uso de su producto.



>>>REFINAMIENTO DE LA CARTERA DE PRODUCTOS

¿Qué es el refinamiento de la cartera de productos?

El Refinamiento de la Pila de Producto (PBR) es una actividad que los equipos Scrum realizan regularmente para aclarar los posibles Elementos de la Pila de Producto (PBI) que se acercan. En Scrum de un solo equipo, por lo general, el equipo Scrum se reúne para uno o más talleres durante un sprint para crear y mantener un Refined Product Backlog .

A continuación, algunas actividades de ejemplo que puede considerar:

- Comprender cuál es el problema correcto a resolver: el propietario del producto cuenta un conjunto coherente de historias con un objetivo y las vincula con el objetivo comercial. El equipo analiza "¿por qué estamos haciendo esto?", "¿Cómo sabemos que estamos resolviendo el problema correcto?". Algunos equipos usan story-maps, mapas de impacto, Speedboat.
- Dividir elementos importantes para debatir y aprender.
- Estimación de elementos para el aprendizaje y la alineación.
- Comprensión de las necesidades del cliente: el equipo analiza "¿Por qué el usuario quiere esto?", "¿Estamos resolviendo el problema correctamente?". Algunos equipos usan un Storyboard (antes / después) y mapas de Pain Gain.
- Aclarar elementos: algunos equipos utilizan Especificación por ejemplo y crean pruebas de aceptación para las historias de usuario; utilizar narrativas de historias con especificaciones de Gherkin, tablas de flujo y / o tablas de decisiones.

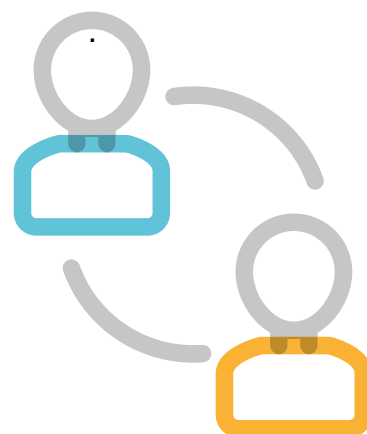
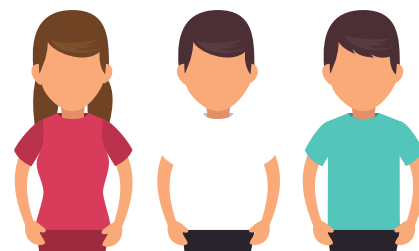


¿Por qué el refinamiento de varios equipos?

PBR de varios equipos es cuando todos los miembros de todos los equipos refinan los PBI juntos sin decidir todavía qué equipo implementará qué elemento.

A continuación, puede encontrar algunos beneficios que el refinamiento de varios equipos puede brindarle: (hay muchos más beneficios, pero eso haría que este blog fuera demasiado largo)

Adaptabilidad a nivel de producto. ¿Por qué?, porque todos los equipos comprenden todos los PBI del Product Backlog, en lugar de que cada equipo comprenda solo "sus" PBI (un subconjunto) del Product Backlog. Si todos los equipos comprenden todas las PBI, entonces la OP puede colocar cualquier PBI que le parezca más valiosa en la parte superior sin ser una limitación para las PBI que un equipo en particular comprende.



Autocoordinación mejorada. ¿Por qué?, porque los equipos mantienen una comprensión amplia de todo el producto y los próximos PBI y, por lo tanto, es más probable que conozcan las "dependencias" entre los PBI.

Medida transparente del progreso a nivel de producto. ¿Por qué?, porque todos los equipos participan en la estimación de todos los PBI y, por lo tanto, hay una velocidad común a nivel de producto, en lugar de una velocidad distinta por equipo que debe combinarse en un total.

Formatos de refinamiento de varios equipos

A continuación te enseñamos las 4 formas de hacer el refinamiento de varios equipos para que los equipos puedan elegir la forma que más les gusta para continuar y mejorar.

Ruleta completa

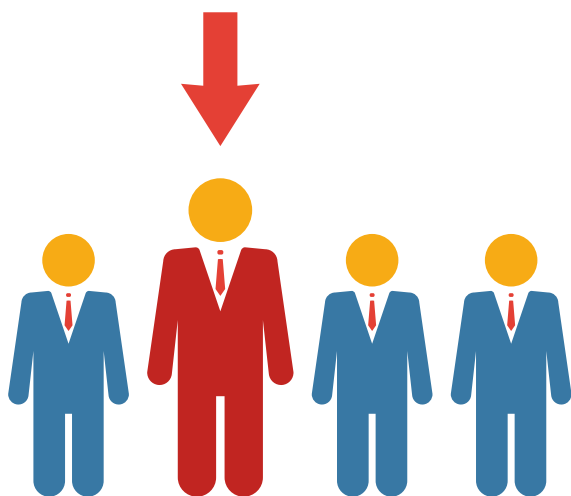
Cada grupo elige un PBI para refinarlo y comenzar a refinarlo en su estación. Después de una casilla de tiempo, todos los grupos pasan a la siguiente estación y continúan refinando donde lo dejó el otro grupo; esto continúa hasta que se refinan los PBI o caduca el plazo del taller. Los grupos pueden usar cualquier técnica que les guste para mejorar.

La ruleta completa da como resultado buenos refinamientos porque los grupos se sienten presionados a dejar su trabajo lo más claro y comprensible posible para el siguiente grupo.



Ruleta parcial

Al igual que la ruleta completa, pero en lugar de que todo el grupo se traslade a otras estaciones, 1 persona se queda en la estación. Esta persona luego le enseña al nuevo grupo lo que se discutió antes de continuar con el refinamiento.



Divergir y fusionar

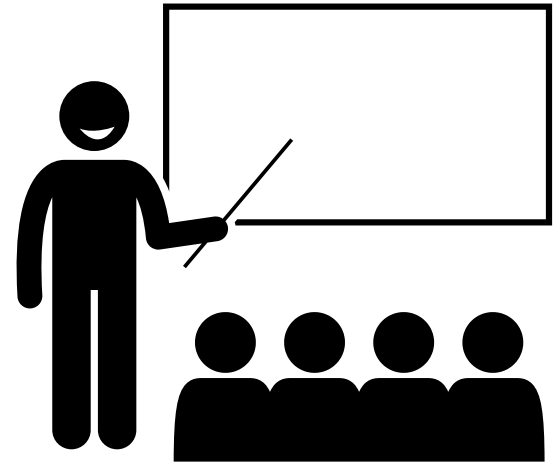
En este enfoque, los grupos realizan una acción de fusión-divergencia después de cada período de tiempo.

Una persona permanece en cada estación y se convierte en maestra. Luego, todos los grupos envían a una persona a cada una de las otras estaciones.

Enseñe de nuevo

Cada grupo elige un PBI y lo refina. Una vez que finaliza el período de tiempo, cada grupo enseña su trabajo hasta ahora a los otros equipos y tiene una discusión de preguntas y respuestas.

Esta técnica no funciona tan bien con una gran cantidad de equipos, porque los profesores de apoyo toman demasiado tiempo para grupos grandes y es más difícil tener una discusión enfocada con mucha gente.



>>>ÁREAS DE ENFOQUE CLAVE

Dentro de cada competencia, una serie de áreas de enfoque brindan una vista más detallada de los conocimientos y habilidades que necesita para dominar esa competencia.

Desarrollo de software emergente

Al resolver problemas complejos, la idea de un diseño inicial detallado se ha reemplazado por un enfoque que fomenta que el diseño surja y cambie dentro de los límites de una arquitectura. En esta área de enfoque, los profesionales podrán describir qué es la arquitectura emergente y cómo se traduce en desarrollo y entrega incrementales. Podrán describir prácticas que "realizan" la arquitectura de forma incremental en un sistema ágil y funcional. Los profesionales comprenderán las compensaciones entre valor, flexibilidad y calidad, y también podrán aplicar técnicas que hagan que el enfoque emergente sea transparente para el equipo, la organización y las partes interesadas.



Gestión de riesgos técnicos

Todos los productos tienen un conjunto inherente de riesgos que gestionar. Estos riesgos van desde la capacidad de entrega hasta los riesgos técnicos asociados con el rendimiento y la seguridad. Esta área de enfoque describe cómo se gestionan los riesgos técnicos dentro de un enfoque ágil. Los profesionales deben comprender qué son los riesgos técnicos y cómo gestionarlos de forma eficaz en un proceso empírico. También deben comprender cómo aplicar prácticas para hacer que los riesgos sean transparentes.



Calidad continua



Trabajar de forma ágil no cambia la importancia de la calidad del producto. Sin embargo, cambia cuándo y dónde se aborda la calidad. Esta área de enfoque describe qué es la calidad y cómo las ideas de Agility y Scrum cambian el enfoque de calidad de un producto. El profesional comprenderá qué es la calidad continua, cómo aplicarla y las prácticas adecuadas para brindar calidad de manera continua. Comprenderán conceptos importantes como la deuda técnica, Test Left y las ideas de las pruebas dirigidas por el usuario.

Integración continua (CI) / Entrega continua (CD)

El aprendizaje frecuente es un concepto fundamental para Scrum. La entrega continua y la integración continua son una colección clave de prácticas que permiten la observación frecuente de las funciones de trabajo. Esta área de enfoque describe el valor de la idea central de que el código siempre debe ser implementable y una comprensión de las técnicas que se pueden emplear para entregar software que resuelva problemas complejos. El practicante comprenderá qué son CI y CD, cómo aplicar estas ideas y qué significan para un proceso empírico y el marco de Scrum.

>>>CONCLUSIONES

Conclusiones finales

El equipo de desarrollo Scrum consiste en profesionales responsables de desarrollar incrementos de producto en cada sprint. La responsabilidad es exclusivamente de ellos, ya que son los únicos que pueden crear estos incrementos.

Es un equipo estructurado y empoderado dentro de la organización para autoorganizarse y gestionar su propio trabajo. Esto se traduce en un equipo óptimo tanto en eficiencia como en productividad.

El equipo de desarrollo en Scrum es una pieza fundamental en este marco de trabajo ágil, ellos son los que harán realidad el proyecto y deben estar sincronizados para aportar el mayor valor durante cada sprint.

¿Te interesa saber más?

Atento al blog de G-Talent, ya que tenemos publicaciones de interés sobre Scrum Development e impartimos cursos donde aprenderás todo lo que necesitas saber sobre esta área.

Curso Scrum Development

Puedes adquirir nuestro curso Scrum Development hoy en descuento, prepárate y domina esta herramienta para convertirte en un experto en esta área.



[Ver Curso Scrum Development](#)

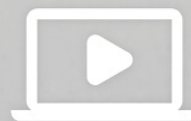


Únete a nuestra Academia y encuentra cientos de cursos, únete a más de 150 mil estudiantes en 15 países de Latinoamérica que hoy estudian en G-Talent.Net.



[Visita G-Talent.Net](#)





Scrum Development

¡Conviértete en un desarrollador profesional en Scrum!



Academia de aprendizaje en línea líder de
Latinoamérica

999 Ponce de Leon, Coral Gables, Miami,
Florida, 33134, United States
info@g-talent.net



G-TALENT.NET

